afterward. The DI container will use your implementation whenever a single instance of the service is requested.

The main disadvantage of this approach is that you still end up with multiple implementations registered; you can inject an `IEnumerable<T>` as before. Sometimes you want to register a service conditionally so that you always have only a single registered implementation.

9.3.3 Conditionally registering services using TryAdd

Sometimes you want to add an implementation of a service only if one hasn't already been added. This approach is particularly useful for library authors; they can create a default implementation of an interface and register it only if the user hasn't already registered their own implementation.

You can find several extension methods for conditional registration in the `Microsoft.Extensions.DependencyInjection.Extensions` namespace, such as `TryAddScoped`. This method checks whether a service has been registered with the container before calling `AddScoped` on the implementation. Listing 9.9 shows how you can add `SmsSender` conditionally if there are no existing `IMessageSender` implementations. As you initially register `EmailSender`, the container ignores the `SmsSender` registration, so it isn't available in your app.

Listing 9.9 Conditionally adding a service using `TryAddScoped`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IMessageSender, EmailSender>();      ❶
builder.Services.TryAddScoped<IMessageSender, SmsSender>();     ❷
```